

Building Efficient Full-text Search Capability for Columnar Database Management System

Larry Luo
IBM Canada Ltd.
larry.luo@ca.ibm.com

Harry Lee
IBM Canada Ltd.
harry.lee@ibm.com

Boris Kuschel
IBM Canada Ltd.
boris.kuschel@ibm.com

Ian Watts
IBM Canada Ltd.
ifwatts@ca.ibm.com

Jason Keirstead
IBM Canada Ltd.
jason.keirstead@ca.ibm.com

Khalid Ahmed
IBM Canada Ltd.
khalida@ca.ibm.com

ABSTRACT

Columnar database management system has been recently gaining more and more popularity in business intelligence, cyber security, enterprise observability and many other types of solutions because of superb performance in analytical query processing.

Creating built-in high performance full-text search capabilities in columnar database management systems has high business value. It simplifies infrastructure and software stack, reduces data duplication. Simplicity means less development and maintenance costs. Removing data duplication avoids data consistency issues from source.

However, building full-text search in a columnar database management system is challenging. Traditional full-text search approaches do not take advantages of the inherent bulk reading/writing mechanism in the columnar database system. They do not perform well in large-scale data sets.

In this paper, we propose an approach combining skipping index and improved inverted index to building efficient full-text search capabilities. The proposed index data structure and construction algorithm can enable high performance full-text search functions in a columnar database management system.

As an example, we describe implementing full text search capability using these techniques in ClickHouse, a prominent columnar database management system, to show the solution can be efficient in time and space and to scale to a very large data set.

CCS CONCEPTS

•Applied computing~Document management and text processing~Document searching•Information systems~Data management systems~Data structures~Data layout~Record and block layout•Information systems~Data management systems~Database management system engines~Online analytical processing engines

KEYWORDS

Inverted Index, Full Text Search, Big Data, Columnar Database, Skipping Index, ClickHouse

ACM Reference format:

Larry Luo, Harry Lee, Boris Kuschel, Ian Watts, Jason Keristead and Khalid Ahamed. 2022. Building Efficient Full-text Search Capability for Columnar Database Management Systems. In *Proceedings of CASCON'22, Toronto, Ontario, Canada, 10 pages*.

1 Introduction

The explosion in the quantity of available and potentially relevant data has been accelerated in recent years. The basic reason for such growth is that there are more tools to easily create and share information than ever before. The cost of collecting and storing information is dramatically reduced by technology advancement.

Database management systems is one of the areas being relentlessly pushed to meet the increasing requirement of the variety of data formats (from structured, through semi-structured, to unstructured), overwhelming data volumes and velocity of big data generation.

1.1 Columnar Database

A powerful technique for making database management systems thrive in the era of ubiquitous big data is Columnar store. Column oriented databases are partitioned vertically — instead of storing

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CASCON'22, November 2022, Toronto, Ontario Canada
Copyright © IBM Canada Ltd., 2022.

data by row, the individual values are stored contiguously in storage by column.

The advantage of a columnar store is that partial reads are much more efficient because a lower volume of data is loaded due to reading only the relevant data instead of the entire row. For applications that need to aggregate data, where only a subset of a table's attributes is needed, column-oriented storage systems can deliver required data faster because data is kept closer together, and seek time is minimized.

Additionally, in column-stores compression is achieved more efficiently than in row-stores because columns have uniform types (ex: all strings or integers). These performance benefits apply to arbitrary access patterns, making them a good choice in the face of unpredictable queries.

1.2 Full-text Search

A full-text search engine is a mechanism for retrieving information that is optimized to efficiently find the data when the user knows only some portion of the data they are looking for.

Because of the rapid or exponential increase in the amount of data that is generated and stored in columnar database management systems, full-text search function becomes extremely important and can tremendously improve user experience. It essentially acts as filters for the sea of data available in a database management system. It allows users to better understand their data, to make better decisions about their business needs. In many business scenarios, users do not know which column has the information they are looking for. This is especially common when multiple columns contain unstructured or semi-structured data. Full-text search is a great tool to help users narrow down focus areas and locate the information they are looking for.

1.2 Inverted Index

Indices, especially inverted indices, are commonly used to implement full text search. As Wikipedia presents,

.... an inverted index (also referred to as a postings list, postings file, or inverted file) is a database index storing a mapping from content, such as words or numbers, to its locations in a table, or in a document or a set of documents (named in contrast to a forward index, which maps from documents to content).

This paper references record-level inverted index, which contains a list of reference to database records for each word or token as shown in Figure 1.

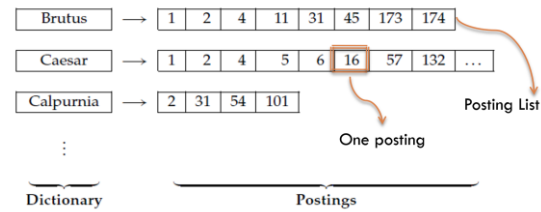


Figure 1: Inverted Index Example

A few things need to be prioritized when building inverted indexes for database management systems:

- search speed
- index compactness
- indexing speed
- update efficiency

One of the challenges of implementing full text search in a columnar database management system is that full text search index has high cardinality in nature. Columnar database has benefits when dealing with low cardinality data. Traditional full-text search approaches adopted by many database management systems do not perform well in columnar databases.

While vertical partition and compression makes column-oriented databases perform well in many application scenarios. However, those data structures pose challenges for searching capabilities, especially full-text search.

This paper will propose some techniques for building efficient full-text search capability for columnar database management system. We will present first, the common reverse index approaches used in several major database management systems first. Then, our proposed approaches and ideas. It will follow the presentation of some examples and related performance experiments. We will finally cover some of the limitations and what could be done next along the same way.

2 Common Approaches in Databases

In this section, we will discuss inverted index implementations in three popular database management systems, MySQL InnoDB, Microsoft SQL Server, and PostgreSQL. A conclusion is provided at the end of the section.

2.1 MySQL InnoDB

Before diving into a full-text search index in MySQL InnoDB, it's necessary to understand InnoDB's table data file and index structure.

2.1.1 Data file and Index Structure.

InnoDB's data files are themselves index files. In InnoDB, the table data file itself is an index structure organized by B+Tree,

When building a FULLTEXT index, InnoDB creates an in-memory cache using red and black tree structure. This in-memory cache structure holds insertions until the cache is full and then batch flushes them to disk (to the auxiliary index tables). The cache improves performance by consolidating duplicate data and reducing IO contentions.

2.1.4 Querying Full-Text Index.

When querying the FULLTEXT index, some mapping operations must be done at query time. This happens because InnoDB table data file itself is the primary index, and full-text index is implemented as secondary index. At query time, InnoDB needs to look up the term dictionary in auxiliary index tables to find DOC_IDS that satisfy search criteria. The qualified DOC_IDS are mapped to primary key values. Then, the actual row data are retrieved from data files using primary key values.

2.2 Microsoft SQL Server

Microsoft SQL Server defines a logical container called "full-text catalog" for storing Full-Text indexes. A full-text index stores information about significant words and their location within one or more columns of a database table.

2.2.1 Data file and Index Structure.

The process of building a full-text index differs from building other types of indexes. Instead of constructing a B-tree structure based on a value stored in a particular row, the Full-Text Engine builds an inverted, stacked, compressed index structure based on individual tokens from the text being indexed using a group of internal tables as shown in Figure 6.

Full-Text indexes are usually split across multiple internal tables that are called full-text index fragments.

	name	parent_object_id	type	type_desc	internal_type_desc
1	fulltext_index_docidstatus_1266103551	1266103551	IT	INTERNAL_TABLE	FULLTEXT_DOCID_STATUS
2	fulltext_docidfilter_1266103551	1266103551	IT	INTERNAL_TABLE	FULLTEXT_DOCID_FILTER
3	fulltext_indexeddocid_1266103551	1266103551	IT	INTERNAL_TABLE	FULLTEXT_INDEXED_DOCID
4	fulltext_avdl_1266103551	1266103551	IT	INTERNAL_TABLE	FULLTEXT_AVDL
5	fulltext_index_docidstatus_98099390	98099390	IT	INTERNAL_TABLE	FULLTEXT_DOCID_STATUS
6	fulltext_docidfilter_98099390	98099390	IT	INTERNAL_TABLE	FULLTEXT_DOCID_FILTER
7	fulltext_indexeddocid_98099390	98099390	IT	INTERNAL_TABLE	FULLTEXT_INDEXED_DOCID
8	fulltext_avdl_98099390	98099390	IT	INTERNAL_TABLE	FULLTEXT_AVDL
9	fulltext_index_docidmap_1733581214	1733581214	IT	INTERNAL_TABLE	FULLTEXT_DOCID_MAP
10	fulltext_index_docidstatus_1733581214	1733581214	IT	INTERNAL_TABLE	FULLTEXT_DOCID_STATUS
11	fulltext_docidfilter_1733581214	1733581214	IT	INTERNAL_TABLE	FULLTEXT_DOCID_FILTER
12	fulltext_indexeddocid_1733581214	1733581214	IT	INTERNAL_TABLE	FULLTEXT_INDEXED_DOCID
13	fulltext_avdl_1733581214	1733581214	IT	INTERNAL_TABLE	FULLTEXT_AVDL
14	fulltext_index_docidstatus_1255675521	1255675521	IT	INTERNAL_TABLE	FULLTEXT_DOCID_STATUS
15	fulltext_docidfilter_1255675521	1255675521	IT	INTERNAL_TABLE	FULLTEXT_DOCID_FILTER
16	fulltext_indexeddocid_1255675521	1255675521	IT	INTERNAL_TABLE	FULLTEXT_INDEXED_DOCID
17	fulltext_avdl_1255675521	1255675521	IT	INTERNAL_TABLE	FULLTEXT_AVDL

Figure 6: SQL Server Full-Text Index

In this example, an internal table named **fulltext_docidmap_1733581214** maps the internal full-text index IDs to the base table full-text key. For performance reasons, integers are used to map each indexed row to the full-text index, even when the row or document ID is another type such as string, text, computed column, and so on.

This mapping is established when the full-text index is defined by the CREATE FULLTEXT INDEX statement.

```
CREATE FULLTEXT INDEX ON table_name
[ ( { column_name
    [ TYPE COLUMN type_column_name ]
    [ LANGUAGE language_term ]
    [ STATISTICAL_SEMANTICS ]
  } [ ,...n ]
) ]
KEY INDEX index_name
[ ON <catalog_filegroup_option> ]
[ WITH [ ( ) <with_option> [ ,...n ] ( ) ] ]
[;]
```

Where KEY INDEX index_name is the name of the unique key index on table_name. The KEY INDEX is required to be a unique, single-key, non-nullable column. Usually, using the smallest unique key index for the full-text unique key provides better performance.

2.2.2 Querying Full-Text Index.

When SQL Server executes a full-text search query in general cases, every full-text search query needs an extra internal JOIN against this internal table in order to map the retrieved rows from the full-text index against the real rows in the base use table.

SQL Server does more performance optimization when the data type for the full-text key is integer. In such case, the fulltext_docidmap_** table is not created, avoiding an extra join at query time. This results in a significant performance gain in many scenarios.

Because Full-Text indexes are split across multiple full-text index fragments, some of these fragments might contain newer data than others. For example, if a user updates some rows in a table and the table is auto change-tracked, a new fragment is created.

Full-text queries need to query each fragment internally and discard older entries.

Increasing the number of full-text index fragments in the full-text index can lead to substantial degradation in query performance. Merging many small fragments into a single larger fragment and removing all obsolete entries from the full-text index can improve query performance.

2.3 PostgreSQL

PostgreSQL has two types of indexes to speed up full text searches. GiST and GIN. GIN index is more expensive to build, but provides better search accuracy and performance. We will focus on GIN index in our discussion.

As PostgreSQL documentation described:

“Internally, a GIN index contains a B-tree index constructed over keys, where each key is an element of one or more indexed items (a member of an array, for example) and where each tuple in a leaf page contains either a pointer to a B-tree of heap pointers (a “posting tree”), or a simple list of heap pointers (a “postings list”) when the list is small enough to fit into a single index tuple along with the key value.”

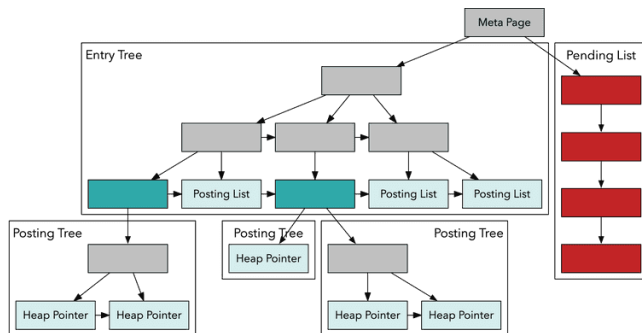


Figure 7: PostgreSQL GIN Index

Because GIN index is implemented using B-tree index structure, GIN indexes are not lossy for standard queries, but their performance depends logarithmically on the number of unique words. Figure 7 illustrates these components of a GIN index.

PostgreSQL defers index updates to improve performance. New data is kept in the special Pending List shown in Figure 7. Certain predefined conditions will trigger the deferred data in pending list being flushed to the main index structure. Flushing a pending list is an expensive operation that often impacts system performance.

2.4 Conclusion

We discussed inverted index implementations in three popular database management systems. All of them directly or indirectly use B-tree as part of the index structure to implement term dictionaries in their inverted index solutions. Those database management systems must support frequent index updates. That is one of the reasons B-tree structure was part of their inverted index implementation.

Assume the height of a B-tree is $h = O(\log n)$, where n is the number of the unique terms stored in the tree. During the search operation, h nodes are read from the disk to the main memory. In addition to the disk reads, the algorithm performs a linear search in every node read from the disk. The time complexity of each linear search is $O(t)$. Thus, the total time complexity of the B-tree search operation is $O(t \log n)$. If the linear search inside each node is changed to a binary search, the total time complexity

of the B-tree search operation becomes $O(\log_2 t \log_2 n)$. However, in practice the disk read operations dominate the time demand of the operation.

As more data is indexed, the size of the term dictionary in inverted index increases exponentially. More random disk accesses are needed to look up a term in B-tree based term dictionary. This has been a big factor impacting full-text search performance in the database management system we discussed in this section.

3 Our Approach

The effectiveness of a full-text search solution is measured by four factors:

- search speed
- indexing speed
- index compactness
- update efficiency

There are competing priorities among them. Keeping the data structures small and compact will improve search speed. But it means sacrificing the possibility to efficiently update them.

Some general-purpose applications try to balance those priorities by giving them equal weight, for example the database management systems discussed in section 2. However, those four factors might have different priorities in an application depending on business requirements. Standalone full-text search engine solutions often give lower priority to update efficiency and put more focus on search speed and indexing speed. This priority arrangement makes a standalone full-text search solution well fit for applications that frequent updates are unlikely to happen or the time required for new changes to be visible is not critical.

Columnar store is the primary choice of OLAP database because columnar store provides high performance gains for analytic queries that need to scan large amounts of data, especially on large tables.

Since the majority of the OLAP use cases in columnar database applications are read-only or with infrequent updates, full-text search functions in columnar databases could adopt a priority strategy similar to standalone search engines. With that in mind, we decided to design a full-text search solution with the following approach.

- Partition full inverted index to create multiple immutable index segments.
- Convert update to delete and insert to avoid updating index data structure directly.
- Minimize term index footprint to make sure the entirety of the term dictionary belongs to an index segment can reside in main memory

- Encode posting list to reduce footprint

We created a solution that accomplishes the above goals in ClickHouse, a fast open-source column-oriented database management system that allows generating analytical data reports in real-time using SQL queries.

3.1 ClickHouse Introduction:

Before diving into the details of our full-text search design, let's briefly review table, data and index structures in ClickHouse.

3.1.1 MergeTree Engine

ClickHouse supports many types of table engine. Table engine determines the data storage strategy and query features. MergeTree family is the most powerful storage engine in ClickHouse, and is the officially recommended storage engine, which supports almost all ClickHouse core functions. MergeTree supports features, such as data partitioning, storage ordering, primary key indexing, data skipping indexing, data TTL, and all ClickHouse SQL syntaxes.

Data in a MergeTree table is stored in "parts". Each part stores data in the primary key order, so data is ordered lexicographically by the primary key tuple. All the table columns are stored in separate column.bin files in these parts. Values in each column are stored in the same order defined by the primary key. Getting values for a row set can be accomplished through iterating a group of columns, then joining the results together.

All parts in MergeTree are immutable. They are only created and deleted, but not modified. In other words, column data and indexes are never changed in a data part.

Our full-text search solution introduces a new type of secondary index, a combination of inverted index and data skipping index into MergeTree.

3.1.2 Primary index structure:

There are two categories of index, dense index and sparse index. B-tree based index commonly used by traditional database management systems is an example of dense index. It records one entry per table row. In contrast, sparse index has index entries for only some of the table rows. As a result, the size of a sparse index could be much smaller than a B-tree based index when indexing the same amount of data.

Primary index in ClickHouse is sparse index, which is best described by the following paragraphs in ClickHouse documentation. Figure 8 shows relationships among table rows, granules and primary index records.

"At the very large scale that ClickHouse is designed for, it is paramount to be very disk and memory efficient. Therefore, instead of indexing every row, the primary index for a part has one index entry (known as a 'mark') per group of rows (called 'granule').

This sparse indexing is possible because ClickHouse is storing the rows for a part on disk ordered by the primary key column(s).

.....

This index design allows for the primary index to be small (it can and must completely fit into the main memory), whilst still significantly speeding up query execution times: especially for range queries that are typical in data analytics use cases. "

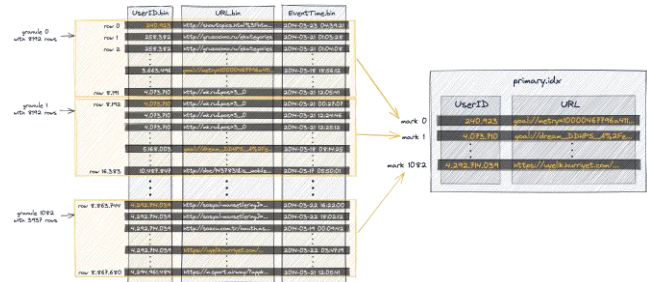


Figure 8: ClickHouse Primary Index Structure

ClickHouse primary index is used for selecting granules other than locating specific rows.

3.1.3 Secondary index:

Like other database management systems, a secondary index can be created in ClickHouse to improve query performance. Unlike traditional database management systems, ClickHouse does not use B-tree index structure. Instead, ClickHouse creates a different structure called "Skip" index for secondary index. These structures are called "Skip" indexes because they enable ClickHouse to skip reading large chunks of data that are guaranteed to have no matching values.

ClickHouse provides several types of skipping index: minmax, set and bloom filter types.

Figure 9 is a graph quoted from ClickHouse documentation that demonstrates a set skipping index defined on a column called my_value, and how it is used to skip data blocks when querying rows whose my_value column contains 125.

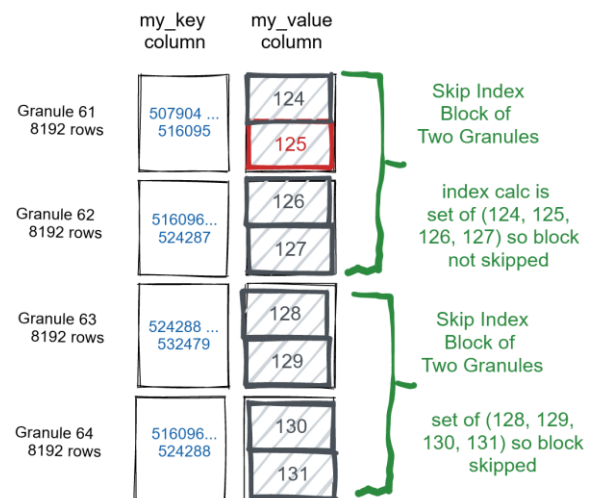


Figure 9: ClickHouse Secondary Index Structure

Skip index can significantly improve query performance in specific circumstances. However, the regular ClickHouse skip index does not work well when indexing high cardinality columns. Certain data distribution in unsorted high cardinality columns could cause skip indexes fail to filter out any data blocks. Skip index becomes pure cost without any benefit in such cases. Token streams in full-text search index have high cardinality in nature. Random data distribution means regular skip index won't be able to speed up query at all.

3.2 Full-text search design:

Our full-text search index is implemented as a new type of skip index with enhancements to handle high cardinality random distribution data streams.

3.2.1 Abstract Architecture:

Regular ClickHouse skip index directly maps summarized column value to granule. As the example shown in Figure 9, index values are summarized results from a SET calculation. They directly map to granules in the relationship represented in Table 1.

This mapping is too coarse to be efficient when the index value has high cardinality. In contrast, our full-text search index combines an inverted index with a skipping index to create a hierarchical mapping structure shown in Figure 10.



Figure 10: Full-text index data mapping

Index Value	Granule
124	Granule 61
125	Granule 61
126	Granule 62
127	Granule 62
128	Granule 63
129	Granule 63
130	Granule 64
131	Granule 64

Table 1: Secondary index data mapping

We proposed the DDL statement shown in Table 2 for creating full-text search index in ClickHouse.

- `INDEX index_name expr`
`(column_name expr`
`[, column_name expr])`
`TYPE FULLTEXT(tokenizer expr)`
`GRANULARITY value`

- `ALTER TABLE [db].name`
`ADD INDEX index_name expr`
`(column_name expr`
`[, column_name expr])`
`TYPE FULLTEXT(tokenizer expr)`
`GRANULARITY value`

Table 2: Full-text index DDL

3.2.2 Full-text Index structure:

Figure 11 depicts full-text search index structure. ClickHouse MergeTree table could have multiple data parts. Full-text index is built as part of the data part. Full-text search query can target a particular data part because each data part has its own full-text index. Each Full-text index consists of two parts, an inverted index, and a skipping index.

3.2.3 Inverted Index:

The full inverted index in a data part is made up of one or more immutable index segments. Each index segment essentially is a fully working inverted index. Index segment represents partition of the full inverted index, and it is fully searchable.

The full inverted index is composed by index segment metadata, term dictionaries and postings lists. Index segment metadata, term dictionary and postings list are stored in separate files on disk. Figure 12 shows the relationship among Index segment metadata file, term dictionary file and postings list file.

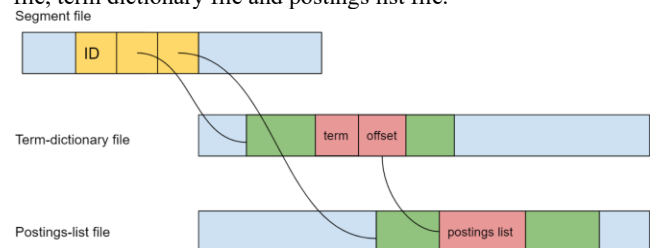


Figure 12: Full-text index data mapping

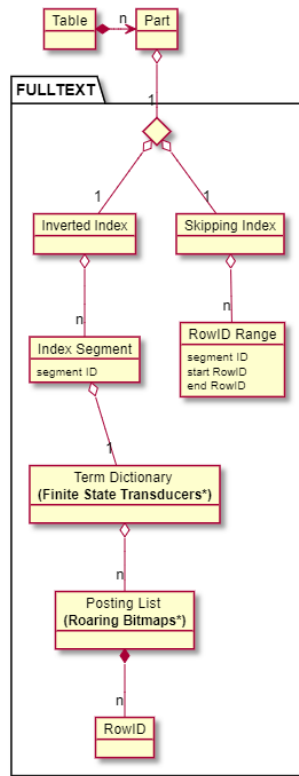


Figure 11: Full-text index data mapping

3.2.3.1 Term Dictionary:

Each index segment contains a term dictionary that is implemented as a Finite State Transducer.

A paper authored by Mehryar Mohri, *"Weighted Finite-State Transducer Algorithms An Overview"* described theories behind Finite State Transducer in detail.

Essentially, Finite State Transducer is a finite-state machine that maps a term (byte sequence) to an arbitrary output. In our case, this output is the postings list location corresponding to the term. Finite State Transducers is a very efficient term dictionary representation. It requires far less RAM than other implementations but has a higher CPU cost during lookup. A low memory footprint is critical for inverted index since an index can easily have many millions of unique terms. FST term dictionary serves as an in-memory term index, that allows search queries quickly to identify whether a term appears in an index segment or not. Traversing FST term dictionary yields the corresponding postings list location offset in postings list file. This greatly reduces the number of disk access required to locate postings list associated with a term.

In our design, the FST term dictionary of an index segment must be loaded into main memory. So, the full inverted index is partitioned in the way that the FST term dictionary of each segment is guaranteed to fit into the main memory and can be bulk read from disk. In memory, the term index is the key to make search queries fast.

3.2.3.2 Postings List:

Postings list is a collection of RowIDs. RowID is a long integer value that uniquely identifies a row within an index segment. We use Roaring bitmaps to represent postings list because Roaring bitmaps are compressed bitmaps. They can be hundreds of times faster according to independent studies.

Roaring BitMaps is a specific type of compressed bitmap index. It represents sets and supports operations such as unions, intersections, differences, and symmetric differences.

A short description from roaringbitmap.org highlights the strength of Roaring bitmaps.

"Bitsets, also called bitmaps, are commonly used as fast data structures. Unfortunately, they can use too much memory. To compensate, we often use compressed bitmaps.

Roaring bitmaps are compressed bitmaps which tend to outperform conventional compressed bitmaps such as WAH, EWAH or Concise. In some instances, they can be hundreds of times faster and they often offer significantly better compression."

3.2.3.3 Skipping Index:

Skipping index in our design is a composition of RowID Range. Each RowID Range represents a group of row data within a particular inverted index segment. Skipping index provides the ability to quickly locate corresponding granules for a Row ID found in a particular inverted index segment. Figure 13 shows the relationship between RowID Range and granule.

Figure 13: Skipping index data mapping

3.3 Indexing process:

Efficiently indexing large amounts of data with millions of distinct tokens is critical for the success of a full-text search feature. Inspired by *"Efficient single-pass index construction for text databases"*, a paper published by Steppen Heinz and Justin Zobel, we created a single pass inversion process that achieves efficiency in both time and space. Figure 14 demonstrates the indexing process.

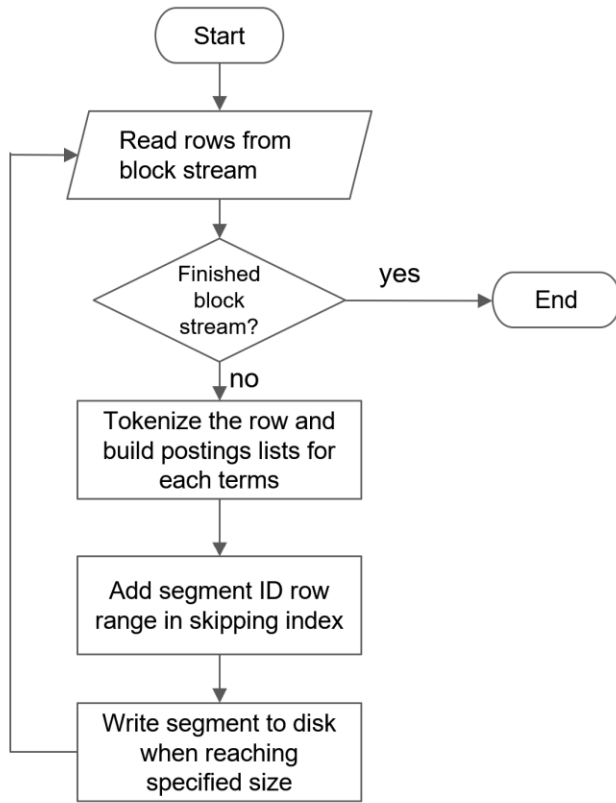


Figure 14: Indexing process

Section 3.3.1 to 3.3.3 list major algorithms used in our single pass inversion process.

3.3.1

Algorithm Build_Inverted_Index(skipping_index, gin_index)

```

(1) row_id_start ← Next_Available_Row_ID(gin_index);
(2) row_id_end ← row_id_start;
(3) need_to_write ← false;
(4) digested_size ← 0;
(5) while block ← Read_Block_From_Pipeline() do
(6)   foreach row in block do
(7)     Gin_Index_Add(gin_index,
(8)       row_id_end,
(9)       row.data,
(10)      row.data_size);
(11)    row_id_end ← row_id_end + 1;
(12)    digested_size ← digested_size + row.data_size;
(13)    if digested_size >
MAX_DIGESTION_SIZE_PER_SEGMENT
(14)      then need_to_write ← true;
(15)    Skipping_Index_Add(skipping_index,
(16)      Get_Current_Segment_ID(gin_index),
(17)      row_id_start,
(18)      row_id_end);
(19)  if need_to_write then
(20)    Gin_Index_Write_Segment(gin_index);
  
```

3.3.2

Algorithm Gin_Index_Add(gin_index, row_id, data, data_size)

```

(1) foreach token in Tokenize(data, data_size) do
(2)   postings_list ← Get_Postings_List(token)
(3)   if postings_list != NIL
(4)     then Postings_List_Add(postings_list, row_id);
(5)   else
(6)     postings_list ← Create_Posting_list(gin_index);
(7)     Postings_List_Add(postings_list, row_id);
  
```

3.3.3

Algorithm Gin_Index_Write_Segment(gin_index)

```

(1) Sort_Postings_Lists_By-Token(gin_index);
(2) postings_list_offsets ← {};
(3) foreach <token, postings_list> in gin_index do
(4)   offset ← Write_Postings_List(postings_list);
(5)   Postings_List_Offsets_Add(postings_list_offsets,
(6)     token, offset);
(7) fst ← Build_FST(gin_index, postings_list_offsets);
(8) Write_FST(fst);
(9) Write_Segment_Meta_Data(gin_index);
  
```

3.4 Query process:

Our index structure is designed with query time efficiency in mind as well. Search query speed is often dominated by disk read operations. Our index structure minimizes the number of disk reads and avoids random access. With our index design, a search query can be executed efficiently using the algorithms depicted in section 3.4.1 and 3.4.2.

3.4.1

Algorithm Gin_Index_Search(skipping_index, gin_index)

```

(1) match_result ← false;
(2) query ← Get-Token_Postinglists_From_Query()
(3) foreach <segment_id, row_id_start, row_id_end> in
skipping_index do
(4)   match_result |= Match_In_Range(query, segment_id,
row_id_start, row_id_end);
(5) return match_result;
  
```

3.4.2

Algorithm Match_In_Range(query, segment_id, row_id_start, row_id_end)

```

(1) intersection_result ← [segment_id, row_id_start,
row_id_end];
(2) foreach postings_list in query do
(3)   intersection_result = intersection_result ∩ postings_list;
(4) return intersection_result != ∅;
  
```

Figure 15 represents the querying process.

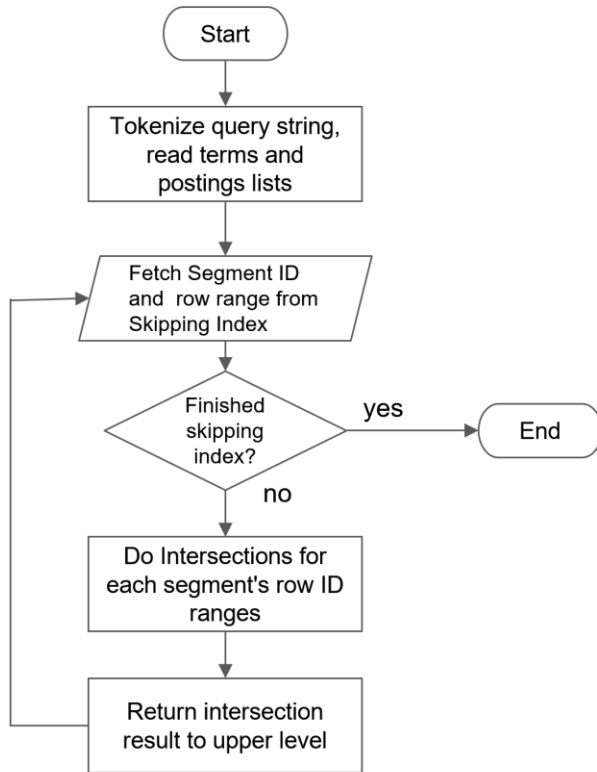


Figure 15: Querying process

3.4 Performance Test Results:

We conducted series of tests using github events dataset `gh-archive-2015-01-text-only.json.gz` from <https://www.gharchive.org/>. The test dataset contains 3000+K rows, which is 1.9G uncompressed data. We tested our implementation with permutations of different tokenizer and ingestion size to get characteristics of indexing time, querying time and memory footprint under different workload.

Figure 16 shows indexing time by ingestion data size for 3 different tokenizers. As the graph shows, indexing time gradually peaked after ingestion data size reached 2100K.

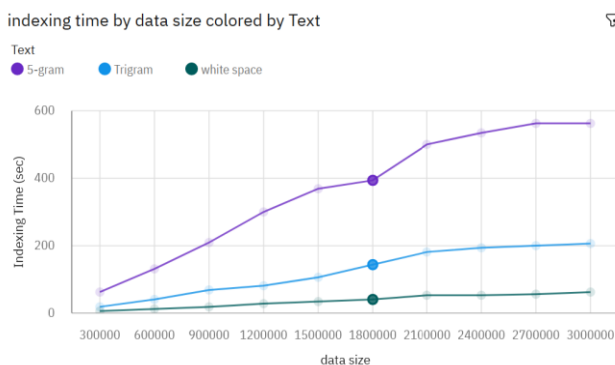


Figure 16: Indexing time by data size (github events)

Figure 17 shows memory usage during indexing by ingestion data size for 3 different tokenizers. As the graph shows, indexing time gradually peaked after ingestion data size reached 1800K.

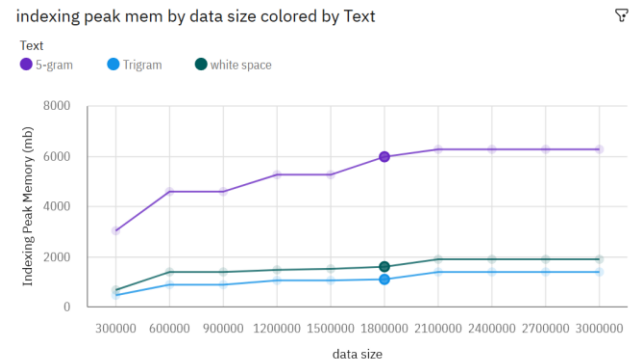


Figure 17: Indexing peak memory by data size (github events)

Another set of performance tests were conducted using a much bigger data set, HackerNews from <https://github.com/HackerNews/API>. HackerNews dataset has 28+ millions rows, which is 12+G in size. Figure 18 shows indexing time by data size; Figure 19 shows indexing peak memory by data size.

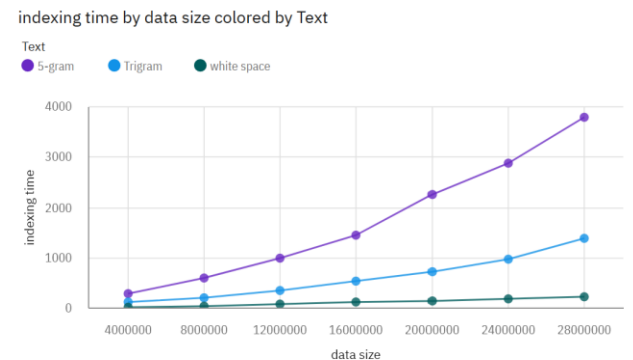


Figure 18: Indexing time by data size (hacker news)

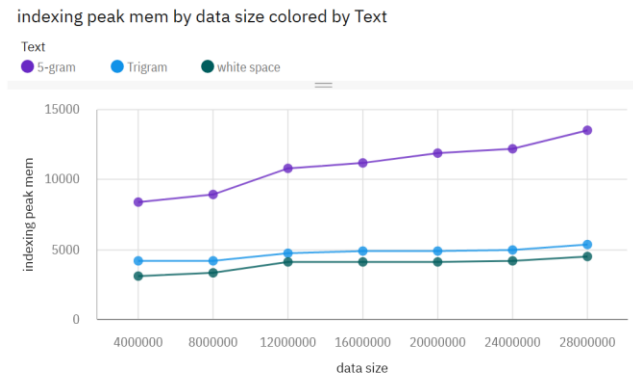


Figure 19: Indexing peak memory by data size (hacker news)

Figure 20 shows search time is consistent when data size grows.

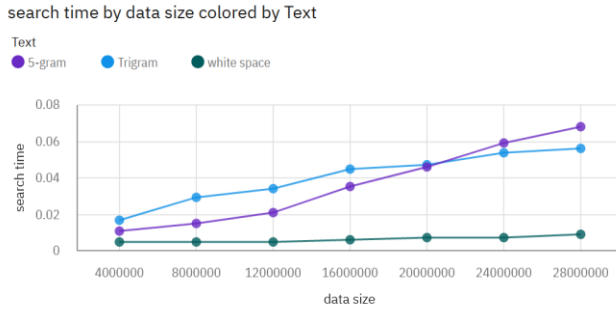


Figure 20: Search time by data size (hacker news)

We compared total time spent when indexing 28+ millions rows hacker news data using trigram tokenizer. Table 9 shows our full-text search solution in ClickHouse is considerable faster.

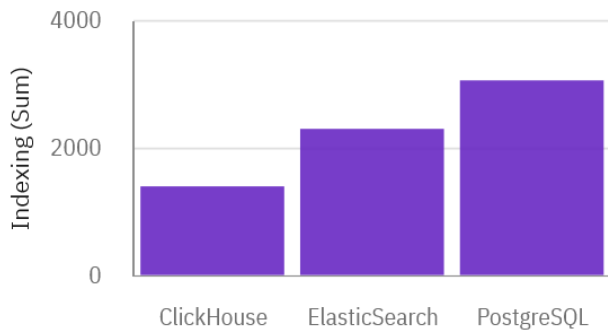


Table 8: Indexing time by solution (hacker news)

4 Future Works

Beside fine tuning and continuously improving our existing design. We also identified several feature enhancements for our future roadmap.

- Providing flexibilities to decouple data ingestion and indexing process
- Enhancing index schema to add statistics information as extra data into term dictionary and postings list to support metadata queries.
- Adding scoring capabilities into search query processing

REFERENCES

- [1] Daniel Lemire, Gregory Ssi-Yan-Kai, Owen Kaser, 2016. Consistently faster and smaller compressed bitmaps with Roaring, Software: Practice and Experience 46 (11). <http://arxiv.org/abs/1603.06549>
- [2] Daniel Lemire, Owen Kaser, Nathan Kurz, Luca Deri, Chris O'Hara, François Saint-Jacques, Gregory Ssi-Yan-Kai, 2018. Roaring Bitmaps: Implementation of an Optimized Software Library, Software: Practice and Experience 48 (4), arXiv:1709.07821
- [3] Heinz, Steffen, and Justin Zobel, 2003. Efficient single-pass index construction for text databases. JASIST 54(8):713–729.
- [4] R. Baeza-Yates and B. Ribeiro-Neto, 1999. Modern Information Retrieval. Addison Wesley.
- [5] Samy Chambi, Daniel Lemire, Owen Kaser, Robert Godin, 2016. Better bitmap performance with Roaring bitmaps. Software: Practice and Experience Volume 46, Issue 5, pages 709–719, <http://arxiv.org/abs/1402.6407>
- [6] Stefan Büttcher, Charles L. A. Clarke etc, 2010. Information Retrieval - Implementing and Evaluating Search Engines. The MIT Press.
- [7] Steffen Heinz, Justin Zobel, Hugh E. Williams, 2002. Burst Tries: A Fast, Efficient Data Structure for String Keys, <https://dl.acm.org/doi/10.1145/506309.506312>
- [8] Stoyan Mihov and Denis Maurel, 2001. Direct construction of minimal acyclic subsequential transducers. Lecture Notes in Computer Science, 2088:217–229.
- [9] MySQL - 14.6.2.4 InnoDB FULLTEXT Indexes | Docs4dev <https://www.docs4dev.com/docs/en/mysql/5.7/reference/innodb-fulltext-index.html>
- [10] GiST and GIN Index Types <https://www.postgresql.org/docs/9.1/textsearch-indexes.html>
- [11] Kerttu Pollari-Malmi. B+-trees <https://www.cs.helsinki.fi/u/mluukkai/tirak2010/B-tree.pdf>
- [12] Microsoft SQL Server Full-Text Search <https://docs.microsoft.com/en-us/sql/relational-databases/search/full-text-search?view=sql-server-ver16>
- [13] ClickHouse MergeTree <https://clickhouse.com/docs/en/engines/table-engines/mergetree-family/mergetree>